

MusicMania

Resumen de organización del proyecto

Desarrollo y Arquitecturas Web (DAW) 2026

Desarrollador: Sebastián Lardelli

Idea central: separar HTML, CSS, JS y assets. En JavaScript usamos namespaces + estado + funciones (un archivo por función), sin APIs externas de juego.

Documento para exposición oral: estructura, conexiones entre módulos, dependencias externas y secuencia paso a paso de los escenarios.

1. ¿Qué es MusicMania?

Juego web de un jugador donde se escucha un tema en versión 8-bit y hay que adivinar la respuesta correcta antes de quedarse sin tiempo o sin vidas. Incluye menú principal, seis categorías jugables y una página de contacto.

Categorías

- **Rock, Pop, Electrónica, Metal:** se elige canción (o artista según reglas) entre opciones de la misma banda.
- **Pelis y Series / Juegos:** se elige el nombre de la obra o del videojuego (sin prefijo “Tema de…”).

2. Estructura de carpetas

La raíz del proyecto agrupa cuatro capas claras. Cada una tiene una responsabilidad y no mezcla estilos ni lógica dentro del HTML.

Carpeta / archivo	RoI
index.html, contacto.html, juego-*.html	Páginas: solo estructura semántica y carga de CSS/JS. Sin lógica inline.
css/	Estilos por pantalla (inicio, juego, contacto) + módulos compartidos (volumen, instrucciones, reset).
js/	Toda la lógica: datos, config, namespaces, estado, funciones y mains de arranque.
assets/	Multimedia local: audios, imágenes, calcomanías e icono. No se descarga de APIs.
README.md	Documentación del proyecto y cumplimiento de consigna.

Detalle de js/

Subcarpeta	Qué contiene
datos/	Catálogo: categorías, niveles por género, opciones de respuesta, datos de contacto.
config/	Une cada página de juego con su lista de niveles e id de categoría.
namespaces/	Objetos vacíos (JuegoLogica, JuegoUi, ControlVolumen...) que agrupan funciones.
estado/	Variables compartidas del módulo (partida, HUD, volumen, media).
funciones/	Una función por archivo, agrupadas por dominio (inicio-*, juego-*, contacto-*).
manifests/	Referencia del orden de scripts (no es un loader automático).
*-main.js	Punto de entrada: DOMContentLoaded e init de cada pantalla.

Detalle de assets/

- **assets/inicio/** — video/audio del menú e icono (favicon).
- **assets/categorias/<Género>/<Nivel>/** — MP3 e imagen/calcomanía de cada nivel.
- **assets/juego/sonidos/** — WAV del temporizador de respuesta (tick, urgente, agotado).

3. Cómo se conectan las pantallas

Menú principal (index.html)

Arranca con **inicio-main.js**. Coordina: media de fondo (video + audio con pantalla “Jugar” por autoplay), nombre del jugador, botones de categoría generados desde **datos-categorias.js**, tabla de récords y control de volumen. Al elegir categoría navega a la página **juego-*.html** correspondiente.

Páginas de juego (juego-*.html)

Las seis páginas comparten la misma lógica (**juego-main.js** + módulos **juego-logica** / **juego-ui**). Lo único que cambia es la config (**juego-config-*.js**) y los datos de niveles (**datos-niveles-*.js**). Al cargar se muestra una pantalla de arranque; el clic en Jugar desbloquea el audio (política de autoplay del navegador).

Contacto (contacto.html)

Formulario validado solo en JavaScript. Al enviar abre el cliente de correo con mailto: (no hay backend ni API de envío). Destino configurable en **datos-contacto.js**.

Flujo de una partida

- Menú: nombre (≥ 3 letras) → se guarda en **localStorage**.
- Clic en categoría → **juego-*.html**.
- Clic en Jugar → empieza el audio del nivel.
- Escuchar → Detener (Espacio/Enter/botón) → elegir 1 de 4 opciones.
- Acierto: puntos según tiempo + calcomanía; fallo o tiempo agotado: derrota (1 vida).
- Récords y volumen persisten en **localStorage**.

4. Módulos JS principales (qué hace cada uno)

Módulo	Responsabilidad
inicio-media	Video/audio del menú, bucle y desbloqueo con gesto del usuario.
inicio-jugador	Validar y guardar el nombre del jugador.
inicio-categorias	Crear botones de categoría y navegar a cada juego.
inicio-records	Mostrar mejores marcas en el menú.
juego-logica	Partida: audio, timers, puntaje, victoria/derrota, opciones.
juego-ui	HUD, pantallas, modales, ecualizador, arranque del juego.
juego-sonidos-timer	Pitidos WAV de la cuenta regresiva al responder.
control-volumen	Slider, mute y aplicación al <audio> activo.
almacenamiento-*	localStorage: jugador, récords, volumen.
utilidades-texto	Normalizar texto, validar nombre, codificar rutas de archivos.
instrucciones-ui	Modal de instrucciones (menú y juego).
contacto-formulario	Validación del formulario y apertura de mailto:.

5. Patrón de arquitectura en JS

No usamos frameworks ni bundlers. El patrón es manual y explícito, pensado para la consigna (ES5 estricto, archivos separados, sin librerías de lógica):

- **Namespace** (objeto vacío) → agrupa funciones relacionadas.
- **Estado** → guarda referencias DOM y variables de la sesión.
- **Función** → un archivo = una responsabilidad clara.
- **Main** → orquesta el init al cargar la página.

Ejemplo: **JuegoUi.init()** obtiene elementos, enlaza eventos y muestra la pantalla de arranque; al hacer clic, **JuegoLogica.comenzarPartidaDesdeMenu()** lee el nombre, reinicia la partida y reproduce el nivel usando rutas de datos-niveles-*.js y opciones de datos-opciones-por-banda.js.

6. Dependencias externas y qué NO hay

El juego es 100 % front-end estático. No hay backend propio, ni base de datos remota, ni llamadas fetch/XHR a APIs de terceros para jugar, puntuar o autenticar.

Elemento	¿Externo?	Detalle
Audios e imágenes	No (locales)	Todo vive en assets/. Se sirven desde el mismo origen (localhost o GitHub Pages).
Lógica del juego	No	HTML5 Audio + JavaScript propio. Sin Howler, jQuery, React, etc.
Persistencia	No (navegador)	localStorage en el cliente (jugador, récords, volumen).
Contacto	mailto: del SO	Abre el cliente de correo del usuario. No hay API de email ni servidor.
Fuente Press Start 2P	Sí (Google Fonts)	Única dependencia de red estética vía @import en CSS. No afecta la lógica.
Enlace GitHub	Sí (link)	Solo un enlace HTML al repositorio. No es una API integrada.

Para la exposición: “No tercerizamos la lógica del juego ni consumimos APIs remotas. Lo único externo es la tipografía de Google Fonts (opcional a nivel visual) y links de navegación (GitHub / mailto).”

7. Tecnologías y buenas prácticas

- HTML5 semántico, CSS3 con Flexbox (sin Grid ni Float como layout principal).
- JavaScript en modo estricto, organizado en muchos archivos pequeños.
- Sin estilos ni scripts inline en el HTML.
- Sin alert: feedback con modales propios.
- Responsive con media queries.
- Publicable como sitio estático (XAMPP local o GitHub Pages).

8. Mapa de conexiones (cómo se habla todo)

index.html → inicio-main → (InicioMedia + InicioJugador + InicioCategorias + InicioRecords + ControlVolumen + InstruccionesUi)

↓ clic categoría

juego-*.html → juego-main → (JuegoLogica + JuegoUi + JuegoSonidosTimer + ControlVolumen) ← datos-niveles + opciones-por-banda + config

↓ assets/categorias (mp3/imagen) + assets/juego/sonidos (wav)

contacto.html → contacto-main → ContactoFormulario → mailto:

localStorage ↔ jugador / récords / volumen (compartido entre pantallas)

9. Secuencia paso a paso de los escenarios

Cómo interactúan las pantallas del proyecto, en orden, para seguir el recorrido completo del usuario y de los módulos.

9.1 Vista global (entre pantallas)

- 1 El jugador abre el sitio y entra al **menú principal** (index.html).
- 2 El menú lee de **localStorage** el nombre, los récords y el volumen (si ya existían).
- 3 Se muestra el menú: media de fondo, campo de nombre, botones de categoría y tabla de récords.
- 4 Si elige una **categoría**: valida el nombre (≥ 3 letras), lo guarda en localStorage y navega a la página juego-*.html correspondiente.
- 5 En el **juego**: se lee de nuevo localStorage, se juega la partida y, al terminar o al ajustar volumen, se actualizan récords/volumen.
- 6 Si en cambio va a **Contacto**: completa el formulario; al enviar se abre el cliente de correo con mailto: (sin usar localStorage).

9.2 Escenario Menú (index.html)

- 1 Se carga la página y corre **inicio-main.js** (DOMContentLoaded).
- 2 Se inicializan, en orden de coordinación: InicioMedia, InicioJugador, InicioCategorias, InicioRecords, ControlVolumen e InstruccionesUi.
- 3 InicioCategorias lee **datos-categorias.js** y crea los botones (Rock, Pop, Electrónica, Metal, Pelis y Series, Juegos).
- 4 InicioRecords lee los récords desde localStorage y arma la tabla.
- 5 ControlVolumen e InicioJugador recuperan volumen y nombre guardados (si hay).
- 6 El jugador hace clic en “Jugar” del menú → InicioMedia desbloquea el autoplay y reproduce video/audio de fondo.
- 7 El jugador escribe su nombre → InicioJugador lo valida (≥ 3 letras).
- 8 Al hacer clic en una categoría → se confirma el nombre, se guarda en localStorage y se navega a juego-*.html.

9.3 Escenario Partida (juego-*.html)

Las seis páginas usan la misma máquina de juego. Solo cambian la config (juego-config-*.js) y los datos de niveles (datos-niveles-*.js).

- 1 Se carga juego-*.html y corre **juego-main.js**.
- 2 Se lee la config de la categoría (id + lista de niveles) y se inicializan JuegoUi, JuegoLogica y ControlVolumen.
- 3 La UI lee nombre y volumen de localStorage y muestra la pantalla de arranque “Jugar”.
- 4 Clic en Jugar → JuegoUi llama a **JuegoLogica.comenzarPartidaDesdeMenu()**.
- 5 La lógica toma el nombre, reinicia la partida, obtiene el nivel actual (datos-niveles) y arma 4 opciones (datos-opciones-por-banda).
- 6 Se reproduce el MP3 8-bit del nivel desde assets/categorias/... y se actualiza el HUD (vidas, puntos, ecualizador).
- 7 El jugador escucha y luego **Detiene** (Espacio, Enter o botón) → se pausa el tema.
- 8 Arranca el temporizador de respuesta (JuegoSonidosTimer con WAV tick / urgente / agotado) y se muestran las 4 opciones.
- 9 **Si acierta a tiempo**: se detiene el timer, se suman puntos según el tiempo restante, se muestra la calcomanía y se pasa al siguiente nivel (vuelve al ciclo audio → detener → opciones).
- 10 **Si falla o se agota el tiempo**: se resta 1 vida. Con vidas restantes se puede continuar; sin vidas, fin de partida y se actualizan récords en localStorage si corresponde.

- 11 Si ajusta volumen/mute durante la partida → ControlVolumen lo aplica al audio activo y lo guarda en localStorage.

9.4 Escenario Contacto (contacto.html)

- 1 Se carga contacto.html y corre **contacto-main.js**.
- 2 Se inicializa ContactoFormulario y se lee el destino del mail desde datos-contacto.js.
- 3 El jugador completa el formulario y lo envía.
- 4 JavaScript valida los campos (sin backend).
- 5 Si hay errores → feedback en modal propio (sin alert).
- 6 Si está todo válido → se abre el cliente de correo del sistema con mailto: (asunto y cuerpo armados).

9.5 Datos compartidos entre escenarios

- 1 El **menú** escribe y lee: jugador, volumen y récords en localStorage.
- 2 El **juego** lee jugador, volumen y récords; escribe récords y volumen.
- 3 **Contacto** no usa localStorage: solo valida y abre mailto:.
- 4 Así menú y juego “se hablan” entre pantallas sin servidor; contacto es un camino aparte.

9.6 Resumen rápido para la oral

Escenario	Entrada	Qué orquesta	Salida / siguiente
Menú	inicio-main	Media, nombre, categorías, récords, volumen	→ juego-*.html o contacto
Juego	juego-main + config/datos	JuegoUi ↔ JuegoLogica ↔ assets	Puntos/vidas → récords en localStorage
Contacto	contacto-main	Validación JS	→ mailto: (sin backend)

Cadena corta: Menú guarda nombre → categoría carga la misma lógica con otros datos → partida usa assets locales → récords/volumen vuelven al menú vía localStorage; contacto es un camino aparte.

10. Frase de cierre para la oral

“Organizamos MusicMania por capas: páginas HTML, estilos CSS, lógica JS modular y recursos locales. Cada categoría reutiliza la misma máquina de juego y solo cambia sus datos. No hay APIs de terceros en el núcleo: el proyecto corre solo con el navegador y archivos estáticos.”